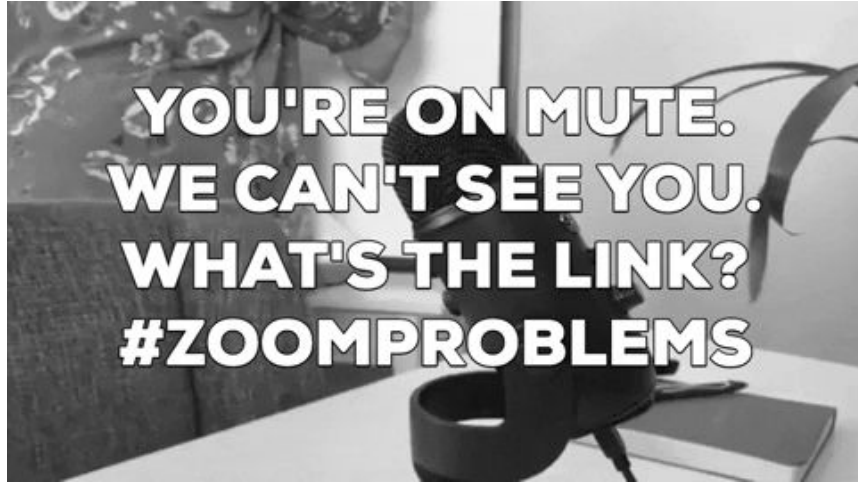


COMS W4156 Advanced Software Engineering (ASE)

September 5, 2023
First day of class!



Important Notice



All of my classes are held on Zoom as part of my ADA accommodations (the course counts as "in person" for ISSO purposes)

If you do not want to take this course on Zoom, then drop the course

Prof. Yang's [W4152 Engineering Software as a Service](#) is an acceptable substitute for most students who are required to take this course

Lecture Notes linked on [Courseworks Calendar](#)

'Live' links into google docs, so possibly changing up to and even during class

Pdfs will be posted on [Courseworks->Files->lecture notes](#) later

Questions and Comments during Class

[Zoom](#), either voice (raise hand) or chat (type whenever you like)

Questions and Comments outside of Class

[Ed Discussion](#) (supports anonymous and private posts)



Are you on the Waitlist?

Student Services Online

► **Wait List**

If you are a Columbia or Barnard undergraduate, I am waiting for your response to the request I've sent several times asking for when you took COMS W3157 Advanced Programming and what was your grade. I will not approve you to enroll in the course until I hear from you (send email to kaiser+4156@cs.columbia.edu).

If you are a CS or CE MS student, you have already been approved unless you joined the waitlist very recently. I'll check the waitlist again later today.

If you are on the waitlist but neither of the above, please wait a bit longer - I'll start admitting non-majors from the waitlist after Thursday's class to fill the space still available.

Agenda

1. Brief explanation of what I mean by 'Software Engineering'
2. Overview of Team Project
3. Overview of this course

Software Developers after
fixing previous bugs with
with new bugs



What is Software Engineering?

Software Engineering !=
Programming

- *Time and Change* affect the sustainability of software, how the *codebase* adapts over the length of its life
- *Scale and Growth* affect the viability of software practices, how the *organization* adapts as it grows and evolves



Time and Change



Google search was a [graduate student project](#) called [Backrub](#) in 1996, written in Java and Python

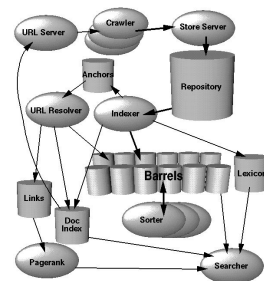
In 1996, Java version 1.0, Python version 1.4

Today, Java version [20.x](#) (many people use earlier [LTS](#) versions), Python version [3.11.x](#) (someone somewhere is probably still using 2.7 even though past [end of life](#))

The original Backrub *could not run today!*

It is unlikely that any other 1996 or even 2006 software written in Java or Python (or any other language) could run as-is today because of *external change*

The platform and execution environment has changed even if the software has not



Software Sustainability

Different from sustainable computing (aka [green computing](#)), although a mandate for green computing would count as an external change

How *painful* is it to modify the software to fit external change?

Could be change in ecosystem: language, compiler, libraries, operating system, hardware

Could be changes in regulations (e.g., [GDPR](#) and other privacy protections)

Could be transient changes due to world circumstances (e.g., [sudden dramatic increase in unemployment insurance claims](#))

Success depends on organizational culture, process and tools



Software Sustainability 2

The codebase has to be modified to fit changed ecosystem, new regulations, the world as is it today and tomorrow rather than when version 1 was released

How do we know whether the software still does what it's supposed to do, how do we keep it working the way it's supposed to, how do we sustain its core functionality?

We test the code to detect the bugs (any deviation from intended core functionality is a bug) and we fix the bugs

And since fixing the code often introduces new bugs, we **test** the code to detect the bugs and we fix the new bugs. And we keep modifying the code and we keep **testing** the code and we keep fixing more and more bugs



Exceptions

A few ancient programs survive

But their code never changes and they are (mostly) exempt from external change



Scale and Growth

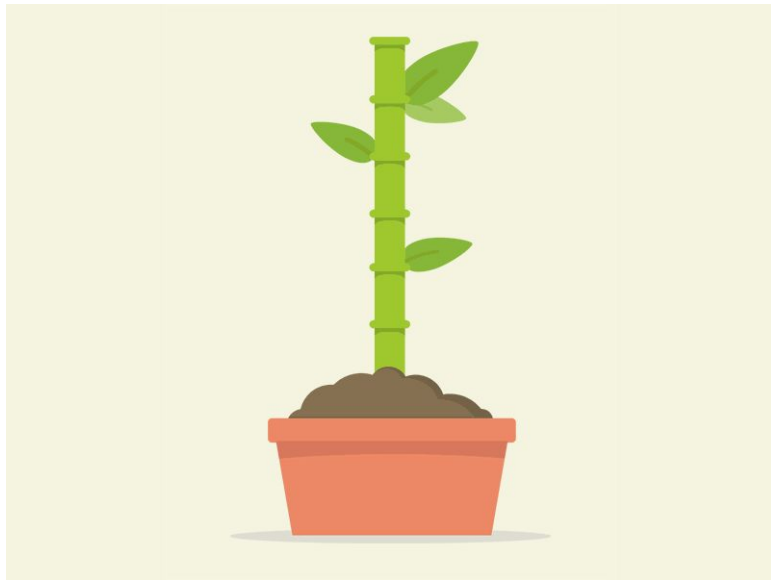
Backrub was written by Larry Page, Sergey Brin and a couple other Stanford graduate students

Their codebase was probably around 10k LOC

At least 25k software engineers now work for Google

Current Google codebase contains billions LOC -
but its still (mostly) in a single monolithic codebase
(all the developers can access all the code)

Elaborate processes, tools and technology enable scaling of both codebase and organization



Human Scalability Reality Check



For course programming assignments where two people work together, it might be feasible to email new versions of the code back and forth

This usually fails by the time the team grows to four people - no one knows what is the current version or who has it

For course projects where four or five people work together using a shared [github](#) (or similar) version control repository, it might be reasonable to turn on [notifications](#) to automatically send email to everyone in the team when anyone pushes modified code

But imagine every software engineer at Google getting email whenever any other software engineer at Google pushes some code (16k commits per day!)

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE.	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAANDS	2 HOURS AGO

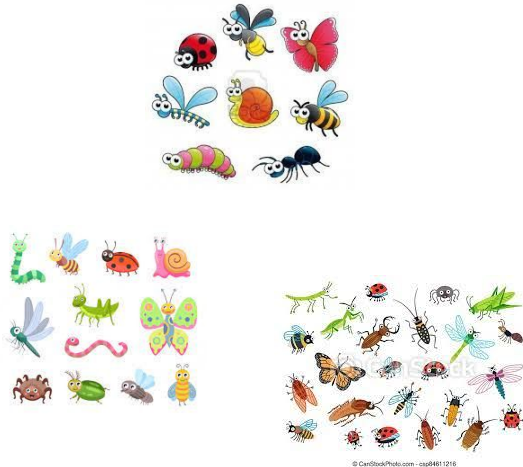
AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

Human Scalability Reality Check 2



Imagine every change in the codebase potentially impacting the core functionality of every other part of the other code ... simultaneously being changed by other software engineers

Imagine all the bugs!



How to Learn Sustainable and Scalable SE

Ideally, the entire class would work on finding and fixing bugs in 25+ year old software consisting of a billion files, with new features constantly being added by tens of thousands of other developers, with four billion active users ... but that's not going to happen



What will happen is a team project (4-5 students per team), where teams propose, develop and test their own project through two iterations

Teams will develop a service during the first iteration and maintain it during the second

Teams will develop a client app that uses the service during the second iteration

The client app must target a user group that is not university students, staff or faculty

Acceptable Target Users

- Elderly (defined for this purpose as age 75 or over)
- Eldercare workers (caregivers for elderly, often non-English speakers)
- Disabled or impaired (e.g., vision, hearing, speech, motor, cognitive)
- Healthcare workers (doctors, medical technicians, nurses, aides, etc.)
- PreK-2 (early childhood) school teachers
- First responders (police, fire, EMT)
- Maybe others with "societal benefits" - ask me *in advance*

Your client needs to know about your target user group
but your service does not,
your service should be useful for a variety
of client apps targeting different user groups



Agenda

1. Brief explanation of what I mean by 'Software Engineering'
2. **Overview of Team Project**
3. Overview of this course



Team Project

Ideally 4 students per team, 5 students allowed (no teams of 1, 2, 3, 6, 7, ...)

Students arrange their own teams, use [Ed Discussion](#) (and/or anything else you want) to find team members

Read [Coping with Hitchhikers and Couch Potatoes on Teams](#) *before* forming team

Every team will have a Mentor who is either an IA or a Leader for this course

Team projects will develop and test a project of their own choice, within constraints



Team Project Constraints

Both the service and the client app must be developed in Java, C++ or Javascript (plus XML/HTML/CSS)

Python is not allowed

The client app must target an acceptable user group that is not university students, staff or faculty (except for individuals who happen to also be members of an acceptable user group)

It should not be possible for someone reading only the service's code (including comments) to guess what the client app does or who is its target user group

The client depends on the service but the service must not depend on the client

Dependencies on third-party software artifacts such as other services, libraries, frameworks, and AI-generated code is ok but your use must be clearly documented



CONSTRAINTS
FORCE
creativity

Acceptable Target Users

- Elderly (defined for this purpose as age 75 or over)
- Eldercare workers (caregivers for elderly, often non-English speakers)
- Disabled or impaired (e.g., vision, hearing, speech, motor, cognitive)
- Healthcare workers (doctors, medical technicians, nurses, aides, etc.)
- PreK-2 (early childhood) school teachers
- First responders (police, fire, EMT)
- Maybe others with "societal benefits" - ask me *in advance*

Remember: Your client needs to know about your target user group but your service does not, your service should be useful for a variety of client apps targeting different user groups



What is a Service?

Service != App

A service provides an application programming interface ([API](#)) for code, not for human users

Services operate independently of any clients (Service != Library)

Clients of a service may be apps or may be other services

When an app consists of [a 'front end' and a 'back end'](#), the back end is not the service, instead it is application-specific code that may use services and libraries

Services, libraries and the differences between services, libraries and apps will be explained in more detail on Thursday

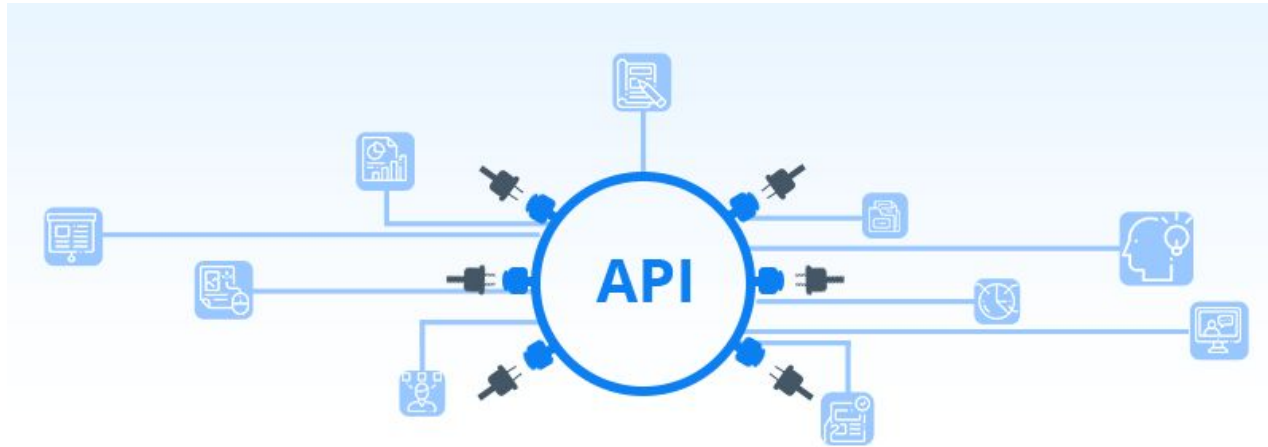


Example APIs

<https://github.com/public-apis/public-apis> and <https://github.com/n0shake/Public-APIs>

<https://www.postman.com/explore/most-popular-apis-this-year>

<https://blog.postman.com/top-30-apis-for-best-developer-experience/>



Team Project Assignments

TEAM FORMATION

PRELIMINARY PROJECT
PROPOSAL

(Team meets with Mentor after
preliminary proposal to receive
feedback towards revised proposal)

REVISED PROJECT PROPOSAL



FIRST ITERATION

FIRST ITERATION DEMO
(for Mentor)

SECOND ITERATION

SECOND ITERATION DEMO
(for Mentor)

DEMO DAY

(for me and any students in class who
want to watch other teams' demos)

“Differential Grading”



Default is for all members of the same team to get the same grade for the project

But every year there are some students who do not do their fair share

Such students will receive a lower grade than their teammates, possibly a much lower grade, based on input from their other team members and their team mentor

Differential grading is applied overall at end of semester, not per assignment

Read [Coping with Hitchhikers and Couch Potatoes on Teams](#) *before* forming team!

Agenda

1. Brief explanation of what I mean by “Software Engineering”
2. Overview of Team Project
3. Overview of this course



Remember What I Said About Testing for Bugs

Real-world codebases are modified repeatedly to fit changing ecosystem, changing regulations, the world as is it today, tomorrow, next year, next decade rather than when version 1 was released

Most software organizations maintaining real-world codebases periodically release new features as well as new bug-fix patches

How do we know whether the software still does what it's supposed to do, how do we keep it working the way it's supposed to, how do we sustain its core functionality?



We **test** the code to detect the bugs and we fix the bugs

And since fixing the code often introduces new bugs, we **test** the code to detect the new bugs and we fix the new bugs

And we keep modifying the code and we keep **testing** the code and we keep fixing more and more bugs

If (non-trivial) software does not have any more bugs to fix, that means no one is using it

This Course Is About Finding Bugs (so they can be fixed)

Testing

Static analysis

Dynamic analysis

Testing

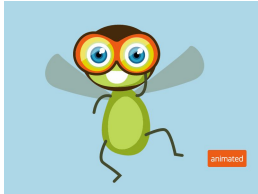
Testing

Testing

Testing

Testing

Testing



If you hate testing (most students do!), and do not want to learn about testing (most students don't!), don't complain - just drop the course

Warning: Your first 5-10 years as a software engineer (until you're a senior engineer and/or a tech lead) will almost certainly include testing other people's code and fixing bugs in other people's code

Visit [Courseworks Home Page](#) for Course Logistics



Next Class

Services are not Apps



Are you on the Waitlist?

Student Services Online

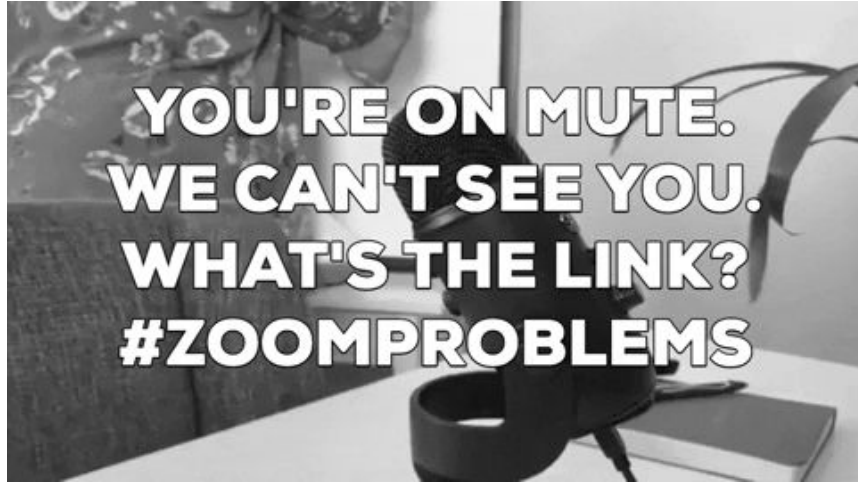
► Wait List

If you are a Columbia or Barnard undergraduate, I am waiting for your response to the request I've sent several times asking for when you took COMS W3157 Advanced Programming and what was your grade. I will not approve you to enroll in the course until I hear from you (send email to kaiser+4156@cs.columbia.edu).

If you are a CS or CE MS student, you have already been approved unless you joined the waitlist very recently. I'll check the waitlist again later today.

If you are on the waitlist but neither of the above, please wait a bit longer - I'll start admitting non-majors from the waitlist after Thursday's class to fill the space still available.

Important Notice



All of my classes are held on Zoom as part of my ADA accommodations (the course counts as "in person" for ISSO purposes)

If you do not want to take this course on Zoom, then drop the course

Prof. Yang's [W4152 Engineering Software as a Service](#) is an acceptable substitute for most students who are required to take this course

Lecture Notes linked on [Courseworks Calendar](#)

'Live' links into google docs, so possibly changing up to and even during class

Pdfs will be posted on [Courseworks->Files->lecture notes](#) later

Questions and Comments during Class

[Zoom](#), either voice (raise hand) or chat (type whenever you like)

Questions and Comments outside of Class

[Ed Discussion](#) (supports anonymous and private posts)



Ask Me Anything